

```
from IPython.display import Image
Image(filename='/content/dsa0201.jpg')
```



```
import cv2

# Read the image
img = cv2.imread('/content/dsa0201.jpg')

# Get the dimensions (height, width, channels)
height, width, _ = img.shape

print(f"Image dimensions: Width = {width} pixels, Height = {height} pixels")
```

Image dimensions: Width = 236 pixels, Height = 359 pixels

```
import cv2
from IPython.display import Image, display

# Convert the image to grayscale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Save the grayscale image to a temporary file to display it
cv2.imwrite('/content/dsa0201_grayscale.jpg', gray_img)

# Display the grayscale image
print("Grayscale image:")
display(Image(filename='/content/dsa0201_grayscale.jpg'))
```

Grayscale image:



```
import cv2
from IPython.display import Image, display

# Define new dimensions
```

```

new_width = 100
new_height = 100
new_dimensions = (new_width, new_height)

# Resize the original image
resized_img = cv2.resize(img, new_dimensions, interpolation=cv2.INTER_AREA)

# Save the resized image to a temporary file
cv2.imwrite('/content/dsa0201_resized_100x100.jpg', resized_img)

# Display the resized image
print(f"Resized image ({new_width}x{new_height}):")
display(Image(filename='/content/dsa0201_resized_100x100.jpg'))

```

Resized image (100x100):



```

import cv2
from IPython.display import Image, display

# Convert grayscale image to binary image using a threshold
# Here, a threshold of 127 is used, and pixels above it become 255 (white), below become 0 (black)
# You might need to adjust the threshold value (127) based on your image.
_, binary_img = cv2.threshold(gray_img, 127, 255, cv2.THRESH_BINARY)

# Save the binary image to a temporary file
cv2.imwrite('/content/dsa0201_binary.jpg', binary_img)

# Display the binary image
print("Binary image:")
display(Image(filename='/content/dsa0201_binary.jpg'))

```

Binary image:



```

import cv2
from IPython.display import Image, display

# Apply a threshold of 170 to the grayscale image
# Pixels above 170 become 255 (white), below become 0 (black)
_, binary_img_170 = cv2.threshold(gray_img, 170, 255, cv2.THRESH_BINARY)

# Save the new binary image to a temporary file
cv2.imwrite('/content/dsa0201_binary_170.jpg', binary_img_170)

# Display the new binary image
print("Binary image with threshold 170:")
display(Image(filename='/content/dsa0201_binary_170.jpg'))

```

Binary image with threshold 170:



```
import cv2
from IPython.display import Image, display

# Apply Canny edge detection to the grayscale image
# The two threshold values (e.g., 50 and 150) are crucial:
# - Edges with intensity gradient above the higher threshold (150) are considered strong edges.
# - Edges with intensity gradient below the lower threshold (50) are suppressed.
# - Edges with intensity gradient between the two thresholds are considered edges only if they are connected to strong edges.
edges = cv2.Canny(gray_img, 50, 150)

# Save the edge-detected image to a temporary file
cv2.imwrite('/content/dsa0201_edges.jpg', edges)

# Display the edge-detected image
print("Edge-detected image using Canny:")
display(Image(filename='/content/dsa0201_edges.jpg'))
```

Edge-detected image using Canny:



```
import numpy as np
import cv2
from IPython.display import Image, display

# Convert image to float32 to avoid clipping when adding noise
img_float = img.astype(np.float32)

# Generate Gaussian noise
# Mean (mu) is typically 0 for noise
mean = 0
# Standard deviation (sigma) controls the intensity of the noise
# You can adjust this value to make the noise more or less prominent
stddev = 25 # A common value, adjust as needed

# Create a noise array with the same shape as the image
noise = np.zeros(img.shape, np.float32)
cv2.randn(noise, mean, stddev)
```

```
# Add noise to the image
noisy_img = cv2.add(img_float, noise)

# Clip values to be within [0, 255] and convert back to uint8
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)

# Save the noisy image to a temporary file
cv2.imwrite('/content/dsa0201_gaussian_noisy.jpg', noisy_img)

# Display the noisy image
print("Image with Gaussian Noise:")
display(Image(filename='/content/dsa0201_gaussian_noisy.jpg'))
```

Image with Gaussian Noise:



```
import numpy as np
import cv2
from IPython.display import Image, display

def add_salt_and_pepper_noise(image, salt_vs_pepper_ratio=0.5, amount=0.04):
    '''Adds salt and pepper noise to an image.

    Args:
        image (numpy.ndarray): The input image.
        salt_vs_pepper_ratio (float): Ratio of salt to pepper noise. (0.5 means equal amounts).
        amount (float): The proportion of the image pixels to be affected by noise.

    Returns:
        numpy.ndarray: Image with salt and pepper noise.
    '''
    noisy_image = np.copy(image)
    # Get image dimensions
    h, w, c = noisy_image.shape
    total_pixels = h * w

    # Number of pixels for salt noise
    num_salt = np.ceil(amount * total_pixels * salt_vs_pepper_ratio)
    # Randomly select coordinates for salt pixels
    coords_salt_x = np.random.randint(0, h, int(num_salt))
    coords_salt_y = np.random.randint(0, w, int(num_salt))
    # Set salt pixels to white (255)
    for i in range(len(coords_salt_x)): # Iterate through the randomly selected coordinates
        x = coords_salt_x[i]
        y = coords_salt_y[i]
        # Ensure x and y are within bounds
        if 0 <= x < h and 0 <= y < w:
            # Apply salt to all color channels for a BGR image
            noisy_image[x, y] = [255, 255, 255] # White

    # Number of pixels for pepper noise
    num_pepper = np.ceil(amount * total_pixels * (1. - salt_vs_pepper_ratio))
    # Randomly select coordinates for pepper pixels
    coords_pepper_x = np.random.randint(0, h, int(num_pepper))
    coords_pepper_y = np.random.randint(0, w, int(num_pepper))
    # Set pepper pixels to black (0)
    for i in range(len(coords_pepper_x)): # Iterate through the randomly selected coordinates
        x = coords_pepper_x[i]
```

```

y = coords_pepper_y[i]
# Ensure x and y are within bounds
if 0 <= x < h and 0 <= y < w:
    # Apply pepper to all color channels for a BGR image
    noisy_image[x, y] = [0, 0, 0] # Black

return noisy_image

# Apply salt and pepper noise to the original image
salt_pepper_noisy_img = add_salt_and_pepper_noise(img, amount=0.02) # Adjust amount as needed

# Save the noisy image to a temporary file
cv2.imwrite('/content/dsa0201_salt_pepper_noisy.jpg', salt_pepper_noisy_img)

# Display the noisy image
print("Image with Salt and Pepper Noise:")
display(Image(filename='/content/dsa0201_salt_pepper_noisy.jpg'))

```

Image with Salt and Pepper Noise:



```

import numpy as np
import cv2
from IPython.display import Image, display

def add_poisson_noise(image, lam=4.0):
    '''Adds Poisson noise to an image.

    Args:
        image (numpy.ndarray): The input image (uint8, BGR).
        lam (float): Lambda parameter for the Poisson distribution. Higher values mean more noise.

    Returns:
        numpy.ndarray: Image with Poisson noise.
    '''
    # Convert image to float and scale it to a suitable range for Poisson noise
    # Poisson noise typically works with counts, so scaling the pixel values is necessary.
    # A common approach is to treat pixel values as \"arrival rates\" or \"expected counts\".
    # Here, we scale the 0-255 range to a smaller range (e.g., 0-lam) and then apply Poisson noise.

    # Convert to float to avoid overflow/underflow issues during scaling
    img_float = image.astype(np.float32)

    # Normalize to [0, 1] range
    normalized_img = img_float / 255.0

    # Apply Poisson noise. The noise generation assumes data is in counts,
    # so we multiply by `lam` before applying `np.random.poisson`.
    # `lam` acts as the mean of the Poisson distribution.
    poisson_noisy_img = np.random.poisson(normalized_img * lam) / lam

    # Scale back to [0, 255] and convert to uint8
    poisson_noisy_img = np.clip(poisson_noisy_img * 255.0, 0, 255).astype(np.uint8)

    return poisson_noisy_img

# Apply Poisson noise to the original image
# You can adjust the 'lam' parameter to control the intensity of the noise
poisson_noisy_img = add_poisson_noise(img, lam=5.0)

```

```
# Save the noisy image to a temporary file
cv2.imwrite('/content/dsa0201_poisson_noisy.jpg', poisson_noisy_img)

# Display the noisy image
print("Image with Poisson Noise:")
display(Image(filename='/content/dsa0201_poisson_noisy.jpg'))
```

Image with Poisson Noise:



```
import numpy as np
import cv2
from IPython.display import Image, display

def add_speckle_noise(image, stddev=0.1):
    '''Adds speckle noise to an image.

    Args:
        image (numpy.ndarray): The input image (uint8, BGR).
        stddev (float): Standard deviation of the Gaussian distribution used for speckle noise.
            Controls the intensity of the noise. (e.g., 0.1, 0.2)

    Returns:
        numpy.ndarray: Image with speckle noise.
    '''
    # Convert image to float32 and normalize to [0, 1] range
    img_float = image.astype(np.float32) / 255.0

    # Generate Gaussian noise with mean 0 and specified standard deviation
    noise = np.random.normal(0, stddev, img_float.shape).astype(np.float32)

    # Add noise to the normalized image and multiply by the image itself
    # Speckle noise is often modeled as I_noisy = I * (1 + N), where N is Gaussian noise
    speckle_noisy_img = img_float + img_float * noise

    # Clip values to be within [0, 1] and then scale back to [0, 255]
    speckle_noisy_img = np.clip(speckle_noisy_img, 0, 1) * 255.0

    # Convert to uint8
    speckle_noisy_img = speckle_noisy_img.astype(np.uint8)

    return speckle_noisy_img

# Apply speckle noise to the original image
# You can adjust the 'stddev' parameter to control the intensity of the noise
speckle_noisy_img = add_speckle_noise(img, stddev=0.1)

# Save the noisy image to a temporary file
cv2.imwrite('/content/dsa0201_speckle_noisy.jpg', speckle_noisy_img)

# Display the noisy image
print("Image with Speckle Noise:")
display(Image(filename='/content/dsa0201_speckle_noisy.jpg'))
```

Image with Speckle Noise:

